

Note To remember: about defining fields and unique constraints

```
class CartItem(models.Model):
    cart = models.ForeignKey(to=Cart, on_delete=models.CASCADE,
related_name='items', default=None)
    product = models.ForeignKey(to=Product, on_delete=models.CASCADE)

    quantity = quantity = models.PositiveSmallIntegerField(
        validators=[MinValueValidator(1)]
    )
```

```
class Meta:
    unique_together = [['cart', 'product']]
*****
```

To define the list of http methods that are accepted to any ViewSet:

```
class CartItemViewSet(ModelViewSet):

    # This list must be in lower case

    http_method_names = ['get', 'post', 'patch', 'delete'];

    def get_serializer_class(self):
        if self.request.method == 'POST':
            return AddCartItemSerialzier

        elif self.request.method == 'PATCH':
            return UpdateCartItemSerializer

        return CartItemSerializer

    def get_queryset(self):
        return CartItem.objects.filter(cart_id =
self.kwargs['cart_pk']).select_related('product')

    def get_serializer_context(self):
        return { 'cart_id': self.kwargs['cart_pk'] }
```

```
*****
```

Note (to remember): we must always check the number of query that are executed before finish from API, we can use Django Toolbar for that.

Note (To remember): the runtime fields must define inside the serializer if we want to send them with body.

```
class AddCartItemSerializer(serializers.ModelSerializer):
    id = serializers.UUIDField(read_only=True)

    product_id = serializers.IntegerField() # Here we set product_id
    # because it is populated only in runtime

    def validate_product_id(self, value):
        if not Product.objects.filter(pk = value).exists():
            raise serializers.ValidationError(f'No product found with value:
{value}')
        return value

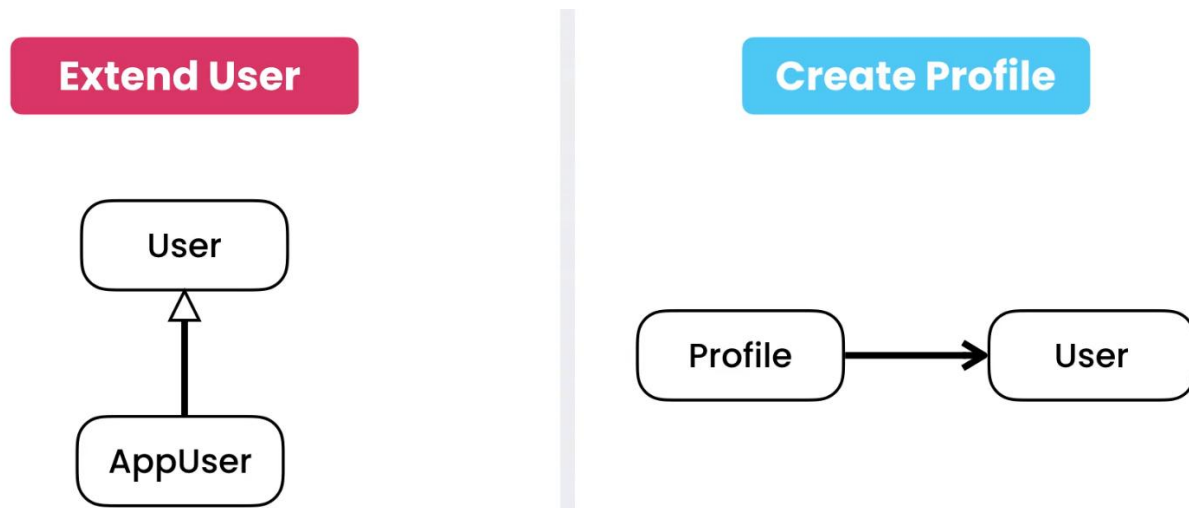
    class Meta:
        model = CartItem
        fields = ['id', 'product_id', 'quantity']
*****
```

Note 1: `is_staff` in `auth_user` for telling if the user can access the admin area.

Note 2: If one of middleware return response then the next one will not be executed (*normal behavior*).

To extend the user model, we have 2-options:

1. **Extend User**: inherit from User class → For storing attributes related to authentication
2. **Create Profile**: use composition and one to one relationship → for storing non-auth related attributes.



To override the custom user model:

```
from django.contrib.auth.models import AbstractUser

from django.db import models

# Create your models here.
class User(AbstractUser):
    email = models.EmailField(unique=True, max_length=150)
```

Then we define inside the settings:

```
AUTH_USER_MODEL='core.User'
```

Then if we have dependency between User-model and Any other independent app, we use:

```
from django.conf import settings

class LikedItem(models.Model):
    user = models.ForeignKey(to=settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)

    content_type = models.ForeignKey(to=ContentType, on_delete=models.CASCADE)

    object_id = models.PositiveIntegerField()

    content_object = GenericForeignKey()
*****
```

In UserAdmin For Admin site to make the email-field appear, then we do:

```
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin

class UserAdmin(BaseUserAdmin):
    add_fieldsets = (
        (
            None,
            {
                "classes": ("wide",),
                "fields": ("username", "usable_password", "password1",
"password2", "email"),
            },
        ),
    )
*****
```

Note: If we create the core User model class late in project may we end with:

1. Delete full database and re-create it
2. Delete the venv folder and recreate it.
3. Run the migrations again

```
*****
```

Note (to remember): the `list_display` in `admin.py` can't accept relation with attributes.

To solve the problem we define the properties as methods inside the `models.py` for the same registered model:

Ex: in `admin.py`

```
@admin.register(models.Customer)

class CustomerAdmin(admin.ModelAdmin):
    list_display = [ 'first_name', 'last_name', 'membership', 'orders' ]
```

And for `models.py`:

```
from django.contrib import admin
class Customer(models.Model):

    # first_name = models.CharField(max_length=255)
    # last_name  = models.CharField(max_length=255)
    # email      = models.EmailField(unique=True)
    user = models.OneToOneField(to=settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)

    class Meta:
        ordering = ['user__first_name', 'user__last_name']

    @admin.display(ordering='user__first_name')
    def first_name(self):
        return self.user.first_name # type: ignore

    @admin.display(ordering='user__last_name')
    def last_name(self):
        return self.user.last_name; # type: ignore
```

Note (From me): Foreach model we create, django will create permissions for add, update, get, delete permissions and store them in `auth_permission`

To create custom permissions that can be used with model-class, we define:

- Define the meta-class inside the target model
- Define permissions list that contains tuples with two fields

```
class Order(models.Model):
    PAYMENT_STATUS_PENDING = 'P'
    PAYMENT_STATUS_COMPLETE = 'C'
    PAYMENT_STATUS_FAILED = 'F'

    PAYMENT_STATUS_CHOICES = [
        (PAYMENT_STATUS_PENDING, 'Pending'),
        (PAYMENT_STATUS_COMPLETE, 'Complete'),
        (PAYMENT_STATUS_FAILED, 'Failed'),
    ]

    placed_at = models.DateTimeField(auto_now_add=True)

    payment_status = models.CharField(max_length=1,
choices=PAYMENT_STATUS_CHOICES, default=PAYMENT_STATUS_PENDING)

    customer = models.ForeignKey(to=Customer, on_delete=models.PROTECT)

    class Meta:
        permissions= [
            ('cancel_order', 'can cancel order') # this custom permission
        ]
*****
```

To adding djoer:

- Run command: *pip install -U djoer*
- In installed apps:

```
INSTALLED_APPS = (  
    'django.contrib.auth',  
    (...),  
    'rest_framework',  
    'djoer',  
    (...),  
)
```

- Updating the urls.py of main project:

```
urlpatterns = [  
    (...),  
    re_path(r'^auth/', include('djoer.urls')), # re_path is from django.urls  
]  
*****
```

To config the auth engine to use JWT:

- In settings.py:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication', # we must add this comma
    ),
}
```

- In settings.py config the headers:

```
SIMPLE_JWT = {
    'AUTH_HEADER_TYPES': ('JWT',),
}
```

- In settings.py add to disable token auth: (TOKEN_MODEL is the default authentication technique for rest_framework but it stores the data in db)

```
DJOSER = {
    'TOKEN_MODEL': None,
    # other settings
}
```

- Update the urls.py of the main project:

```
urlpatterns = [
    (...),
    re_path(r'^auth/', include('djoser.urls')),
    re_path(r'^auth/', include('djoser.urls.jwt')),
]

*****
```


To override the user create serializer to include our fields that contains the data about our customer:

- In settings.py:

```
DJOSER = {
    'TOKEN_MODEL': None,
    'SERIALIZERS': {
        'user_create': 'core.serializers.UserCreateSerializer',
    },
    # other settings
}
```

- Then we create serializers.py in core-app, with the content:
 - Note: here we inherit every thing from Base.Meta for serializer meta class

```
from djoser.serializers import UserCreateSerializer as Base

class UserCreateSerializer(Base):

    class Meta(Base.Meta):
        fields = ['id', 'username', 'email', 'password', 'first_name',
            'last_name']
*****
```

To create our profile api, we go to store-app:

- Inside the serializers.py:

```
class CustomerSerializer(serializers.ModelSerializer):
    user_id = serializers.IntegerField()
    class Meta:
        model = Customer
        fields = ['id', 'user_id', 'phone', 'birth_date', 'membership']
```

- Inside the views.py of store-app:

```
class CustomerViewSet(CreateModelMixin, UpdateModelMixin, RetrieveModelMixin,
GenericViewSet):
    queryset = Customer.objects.all()
    serializer_class = CustomerSerializer
```

- Inside the urls.py of store-app:

```
router.register('customers', views.CustomerViewSet)
*****

https://django-rest-framework-simplejwt.readthedocs.io/en/latest/settings.html
*****
```

To override the JWT lifetime:

```
SIMPLE_JWT = {
    'AUTH_HEADER_TYPES': ('JWT',),
    "ACCESS_TOKEN_LIFETIME": timedelta(days=30),
}
```

```
*****

http://localhost:8000/auth/jwt/create
*****
```